Machine Learning for the extraction of RTN parameters

Guillermo Martín Romero
Instituto de Microelectrónica,
CSIC,
Sevilla,
email: guille.martin.romero@gmail.com

In this paper a new method of obtaining the essential parameters of RTN fenomena based on techniques of Machine Leaning. The principles of the clustering method known as K-Means and how it can be implemented in the detection of RTN signal are treated in this paper. Some techniques for automatic choice of the correct number of levels in RTN signal are explained in detail, these are Silhouette Method and Gap Statistics Method. Moreover, these techniques have been combined with K-Means method to create a routine that could correctly detect RTN. Furthermore, a comparison between using Maximum Likelihood method (MLE onwards) or K-Means method in the analysis of RTN signal is shown. This comparison is going to be done by the application of the routines based on both methods to different sets of current data that contains RTN signal and observing the clean current which provide each code. Finally, it is going to be shown how preprocessing current data by a low pass filter before applying K-Means improve the results in a huge manner.

Keywords: Random Telegraph Noise (RTN), Clustering, Machine Learning

1 Introduction

Random Telegraph Noise (RTN) is a type of electronic noise affecting semiconductor devices. It is caused by charge traps in the device's gate dielectric, which capture and release charge carriers. This results in random, abrupt fluctuations on current or voltage between two levels. If there exists n traps, 2^n levels could be observed in current or voltage signal. RTN's effects include performance variations, reliability issues, signal integrity problems, and power consumption changes so dealing with RTN is essential for ensuring the functionality and longevity of modern electronic devices.

The essential parameters in RTN signal are capture and emission times, t_c and t_e respectively, as well as amplitude ΔI or ΔV. Emission and capture times are stochastic variables with mean values τ_e and τ_c. They represent the time it takes for the trap to emit or capture charge. The value of ΔI depends on factors as the positions of the trap in the channel (if it is centered in the channel it will have a larger impact than if it close to the drain or source). In Fig.1 it is shown an example of a RTN signal showing its parameters.

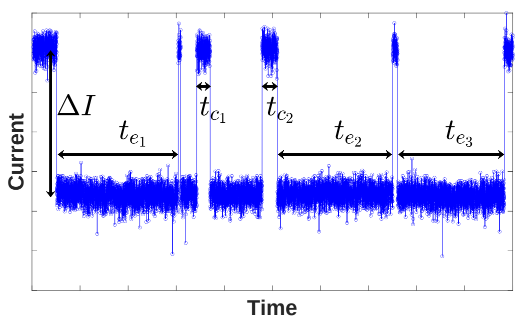

Fig. 1 An ilustrative measure of a drain current from a device with one defect. Times t_{e_1}, t_{e_2} and t_{e_3} are some emission times that we could measure in this trace whereas t_{c_1} and t_{c_2} are capture times. ΔI is the distance between levels in this two-levels RTN measurement.

The problem of obtaining these parameters has become essential and has been covered in different ways. One of them is using Maximum Likelihood Estimation (MLE) and, what we show in this paper, using a Machine Learning Method known as K-Means. First of all, these methods are going to be explained in detail in the next sections.

2 Maximum Likelihood Estimation (MLE)

This method tries to estimate the parameters of a determined probability distribution using some current data. The procedure consists in grouping measured data in an histogram, where the distribution of a RTN signal can be approximated as the sum of Gaussian distributions where the background noise will be indicated due to the the deviation, so the probability distribution of the intensity I assume the form

$$f(I;\theta) = \sum_{j=1}^{N} A_j \exp \frac{(I - I_j)^2}{2\sigma^2}. \tag{1}$$

Here θ represent the set of parameters of our distribution, that is to say, N is the number of levels which conform the signal, I_j symbolize the levels of the RTN, A_j illustrate how populated is each level and σ is the normal deviation of the gaussians.

In the case that we have M samples of a current, the joint density function of all observations, $F(\theta)$, adopt the form

$$F(\theta) = \prod_{j=1}^{M} f(I_j;\theta). \tag{2}$$

In this context, this function is also called the likelihood function of parameters θ.

The MLE method consists in find the values for the parameters of θ that maximizes $F(\theta)$. It can be found by the calculation of the partial derivative with respect to each variable equalized to zero. Once levels are found, for getting the clean trace each point is related with the closest level, then it is clear how to save the points where the current change it high and get capture and emission times.

However, this method present some problems which become evident when some traces are observed at first glance. An example of incorrect fit is shown in Fig.2. MLE identifies four levels but it is clear for our eye that there exists a few more, specially in the interval before forty seconds that, at least, we can identify two

Version 1.18, February 10, 2026

levels. One rigorous analysis about this type of failures indicates that apart from MLE inaccuracies another source of errors is the step in which the histogram is done. The automatic election of the width of the bars is essential in some cases where RTN levels are close (the amplitude is small compared with the range of data) and it is an heuristic challenge to deal with when applying MLE method.

The repetition of fits like this one makes it necessary to find another way to get the approximate current trace and that is where Machine Learning methods take place. In the next section, we are going to expose the fundamentals of a clustering method that can be used in this type of problems.

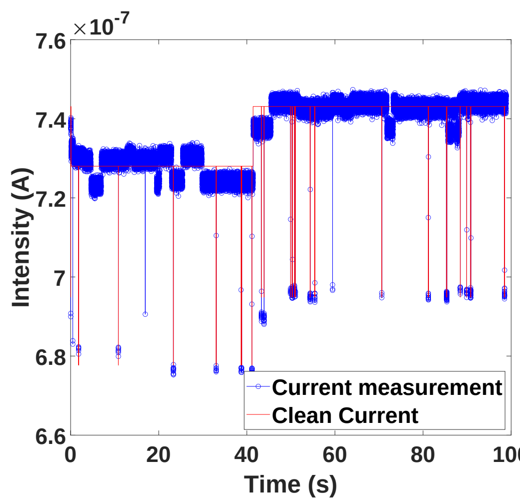

Fig. 2 An example of a fit done by the application of MLE method to ilustrate how it can fail in the detection of levels.

3 Machine Learning Methods

Machine learning is a powerful branch of artificial intelligence that has revolutionized the way we analyze and process data. It enables computers to automatically learn and improve from experience without being explicitly programmed. This technology has found extensive applications in various fields, including data processing. In this work we show how could we use one Machine Learning algorithm to solve our problem of fitting RTN current traces.

Machine learning algorithms can be categorized into two main types: supervised and unsupervised learning. In supervised learning, algorithms are trained on labeled datasets, where each input data point is associated with a known output. The algorithm learns to make predictions based on this labeled data. This is commonly used in tasks like classification and regression.

Unsupervised learning, on the other hand, deals with unlabeled data. The goal is to discover patterns or structures within the data. Clustering and dimensionality reduction are typical applications of unsupervised learning techniques. It is clear that, for our purpose, we must use this type of learning. In particular, it is going to be used a clustering algorithm known as K-Means, which is explained in detail below.

3.1 K-Means. The K-Means method is a fundamental clustering technique in machine learning and data analysis. Its primary objective is to group data points into clusters based on their similarity. This method is widely used for various applications, including customer segmentation, image processing, and data compression.

K-Means is known for its simplicity and efficiency in handling large datasets. It relies on the notion that data points within the same cluster are closer to each other than to data points in other clusters. By partitioning the data into clusters, it becomes easier to analyze and draw insights from complex datasets.

This method does not require a training. It works following the next steps.

(1) Introduce a set of current's measurements and a number of levels in which group the data, we call them 'clusters'.
(2) The algorithm starts by randomly selecting the positions of the current levels it is seeking and groups the measured data into different levels based on proximity (Euclidean distance).
(3) Once we have the set grouped, we calculate the mean value in each cluster (centroid) and select them as the new levels from which to re-group the data.
(4) Repeat this steps until convergence is achieved. When algorithm finish, the position of the current levels will be the centers of each cluster.

After grouping data, clean current is built by assigning, for each measured current data point, a corresponding clean current value equal to the mean value of the cluster to which it belongs.

It is important to notice that the algorithm can converge to different local optima based on the initial selection of cluster centroids, potentially leading to suboptimal results. Another limitation is that K-Means assumes that clusters are spherical or equally sized, which may not hold true for all types of data. Furthermore, the requirement to specify the number of clusters, 'k,' in advance can be challenging when there is no prior knowledge about the data, and selecting an inappropriate 'k' value can impact the quality of clustering. To maximize its effectiveness, careful consideration of factors such as the choice of 'k,' appropriate initialization methods, and post-clustering evaluation techniques should be employed.

In this context, we will focus on the next subsection, which is the automatic selection of the correct number of clusters that best fit the curve. Specifically, we will concentrate on methods known as Silhouette and Gap Statistics. We will introduce their mathematical foundations and present some algorithmic results using either of them.

4 Automatic selection of clusters' number

In this subsection, two different ways of selecting the appropriate clusters' number are going to be explained. This step is essential in the creation of an automatic algorithm to detect RTN traces. There are some other ways of achieving our goal such as Calinski-Harabasz Index, Elbow Method or Dendrogram Analysis. However, it is going to be show the two ways that provide the best results.

4.1 Silhouette Score. The Silhouette Score is a metric used to evaluate the quality of clusters in a clustering analysis. It provides a measure of how similar each data point in one cluster is to the data points in the same cluster compared to other clusters. A higher Silhouette Score indicates that the clusters are well-separated, while a lower score suggests that data points may have been assigned to the wrong clusters or that the clustering is not well-defined.

Connecting with our algorithm, we are going to establish a maximum number of clusters among which to search for the optimal number of clusters. This is an heuristic decision that could be modified in the model. Normally, this number is set between ten and twenty but this range could be modified depending on the problem we are covering.

Once we have reached this point and before showing the results of the algorithm combining KMeans and Silhouette Score, it is necessary explain the theoretical notions of this metric. The steps are mentioned below.

(1) Select a number of clusters in the range we are considering and group the measurements in clusters.
(2) For each data point in the dataset, compute two values:

- **a(i)**: The average distance from the data point to all other data points in the same cluster. This measures the cohesion within the same cluster.
- **b(i)**: The smallest average distance from the data point to all data points in a different cluster, where the data point doesn't belong. This measures the separation from other clusters.

(3) For each data point, calculate its Silhouette Score (s(i)) using

$$\mathrm{s(i)} = \frac{\mathrm{b(i)} - \mathrm{a(i)}}{\max\{\mathrm{a(i)}, \mathrm{b(i)}\}}. \quad (3)$$

(4) The Silhouette Score for each point in our data will be found among -1 and +1, where each one have the next meanings:

- -1 indicates that the data point is likely placed in the wrong cluster.
- 0 indicates that the data point is on or very close to the decision boundary between two neighboring clusters.
- +1 indicates that the data point is well-clustered and far away from neighboring clusters.

(5) Calculate the average Silhouette Score, $\mathrm{S_T}$, for all data points as

$$\mathrm{S_T} = \sum_{i=1}^{N} \frac{s(i)}{N}, \quad (4)$$

where N is considered as the total number of current points. This overall score provides an estimate of the quality of the clustering.

(6) Save, for each number of clusters in the range we are considering, its Silhouette Score and the best fit is the one with the higher score (closest to +1).

A issue with this method is that it requires a cluster range that starts at a minimum of two; therefore, it is not recommended for datasets with a clear absence of a true underlying cluster structure, as it may collapse and select the maximum number of clusters within the specified range. Another problem could be the amount of time it takes to run the algorithm for very large datasets. Once we finish discussing the next method, we will provide a comparison between both methods.

4.2 Gap Statistics. Now, Gap Statistics method for determining the optimal number of clusters in experimental data by comparing the clustering results for different values of k (the number of clusters) and identifying the point at which adding more clusters no longer significantly improves the clustering quality.

The basic idea of Gap Statistics is that it compare the quality of a clustering solution for a specific k value to the expected quality of clustering that would be obtained from a random dataset with no inherent structure. If the clustering quality of the real data (with a certain k) is significantly better than the expected quality of random data, it suggests that k is a good choice.

Gap statistics are founded on the concept of within-cluster dispersion and the expectation under randomness. Here are explained some mathematical concepts which are essential to understand this method:

- **Within-Cluster Dispersion (WCSS)**: For a given k, you calculate the sum of squared distances between data points and their assigned cluster centers (WCSS). Lower WCSS values indicate tighter clusters and better clustering quality.

- **Expected WCSS** ($\mathrm{WCSS_{rand}}$): You generate B random datasets that have the same dimensionality and range as your real data. For each random dataset, you perform the same clustering procedure and calculate the $\mathrm{WCSS_{rand}}$ for each k.

- **Gap Statistics Gap(k)**: Gap statistics then calculate the gap for each k value as the logarithm of the expected $\mathrm{WCSS_{rand}}$ minus the logarithm of the actual WCSS. The gap measures the relative difference between the clustering quality of the real data and the random data. It follows

$$\mathrm{Gap(k)} = \log(\mathrm{WCSS_{rand}}) - \log(\mathrm{WCSS}), \quad (5)$$

where $\mathrm{WCSS_{rand}}$ and WCSS are evaluated for k clusters.

Implementing Gap Statistics is similar to Silhouette and the steps are

(1) Establish a maximum number of clusters among which to search the correct fit (heuristic step).
(2) Calculate WCSS for each k.
(3) Generate random datasets with the same nature as initial measurements and perform k-means clustering on each.
(4) Calculate $\mathrm{WCSS_{rand}}$ for each k in the random datasets.
(5) Compute the gap values for each k using (5).

The last step is to determine the optimal k based on the gap values. The optimal number of clusters is typically chosen as the value where the gap is maximized or reaches a plateau.A larger gap value indicates that the clustering result for a specific k is significantly better than what you'd expect from random data.

The gap statistic provides quantitative insight into the number of clusters that best represent the structure of your data. However, it's essential to interpret the results in conjunction with domain knowledge and the specific context of your analysis. It is a valuable tool for cluster validation and can help you avoid overfitting or underfitting in your clustering models by guiding you towards the most appropriate number of clusters for your dataset.

5 Comparison

A critical point in the moment of selecting one of these methods for the algorithm automation of KMeans is the time it takes for running using each one and it is shown in the next table.

	5k points	**50k points**
Silhouette Score	4-5 sec	15-20 sec
Gap Statistics	4-5 sec	4-5 min

Table 1 Comparison between running times of both methods of obtaining the correct number of clusters in the current trace for some amounts of measurement points.

Observing Table 1, we can see how, as the number of measurements increases, the loading time for both algorithms also increases. This is something that is expected; however, the growth in loading time for the Silhouette Score method spikes, while the Gap Statistics method maintains a reasonable running time.

Next, we will present a precision comparison among the Maximum Likelihood Estimation (MLE) method, KMeans combined with Gap Statistics, and KMeans with Silhouette Score. To compile this comparison, approximately two hundred random traces, which mostly exhibit the visible human-eye-observable Random Telegraph Noise (RTN) effect, have been utilized. All three algorithms have been applied to these traces, and the results of each fit have been visually assessed and categorized as either correct or incorrect. Precision is calculated as the ratio of the number of correct fits to the total number of traces.

	Precision (%)
MLE	72
KMeans + Gap Statistics	81
KMeans + Silhouette Score	84

Table 2 Precision comparison between MLE method and the choices to improve RTN fitting.

As it is shown, the KMeans algorithm combined with either Gap Statistics and Silhouette Score provides better results than MLE. Furthermore, the difference between applying Gap Statistics or Silhouette Score is so minimal and it becomes relevant in some traces where human eye has a hard decision about the correct number of levels. However, a big problem in using Silhouette Score is that it is not able to fit traces where only one level is present, it could need a pre-algorithm to run Silhouette Score if it detects more than one level.

In Fig.5 and 6, a visual comparison between this three algorithms fitting some traces is shown. It is first important to highlight that, as previously mentioned, the results provided by the KMeans methods combined with Gap Statistics and Silhouette Score are predominantly identical. It is unlikely to find instances where these adjustments do not precisely correspond to each other. In both Fig.5 and 6, it can be observed how the KMeans algorithm better fits the current trace than in the case of MLE. These examples have been chosen because they clearly illustrate shortcomings of the MLE method. The Fig.5 addresses the failure example of the maximum likelihood method shown in Fig.2.

6 Data Preprocessing

One of our initial challenges to address in this paper was the acquisition of emission and capture times for the various traces. To achieve this, the method described in this document beforehand involves, once we have grouped the data into their respective clusters (closest current levels), constructing the clean current by associating each data point in our measurements with the centroid value of the cluster to which it has been grouped. This method appears to be effective, but when we consider it more generally, we see that it presents a fundamental issue: its performance in traces where the noise is on the order of the separation between levels. This fact could cause some data points that, a priori, seem to be encompassed within one level, to be grouped in another cluster simply because the distance to that level is shorter than what it should visually correspond to due to noise.

The chosen approach for processing involves applying a low-pass filter to the raw initial data. The sampling period of our data acquisition device is two milliseconds. It is essential to justify the cutoff frequency above which we will apply the filtering to minimize information loss regarding our trace (possible very rapid RTN events). Consequently, a cutoff frequency of 125 Hz has been selected, which corresponds to four measurement periods of our device. We would lose frequencies higher than this, but even if they exist, with only four data points, we would have very limited statistical information to accurately determine emission and capture times. In Fig.3 it is shown the effect of applying a low-pass filter to a raw current. It is evident how noise in the signal is reduced.

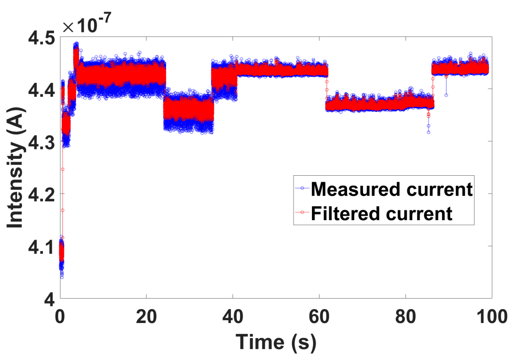

Fig. 3 Comparison between the filtered current provided by the application of the low-pass filter previously commented and the measured current.

By applying the low-pass filter discussed earlier (easily implementable in Python using some of its libraries such as 'scipy'), a significantly cleaner trace will be obtained, with a substantial reduction in noise levels. What will be observed is that the points in the various clusters tend to cluster more closely around their centroids, reducing the within-cluster deviation. This, in turn, allows for a more precise assignment of each point to its corresponding level.

For the example in Fig.3, we will verify how the signal would behave if we do not apply the low-pass filtering as a preprocessing tool, comparing it with what it would look like if we do apply the filtering. This is shown in Fig.4. We can observe that the detection of levels and their positions remains unchanged when applying one method or the other. Where we do see a difference is in obtaining emission and capture times in the clean current. The algorithm, when filtering is applied, better fits the shape of the raw current and is able to organize the points into the clusters that should be closer. In the case of applying KMeans without prior processing, level jumps are detected, which actually result from noise.

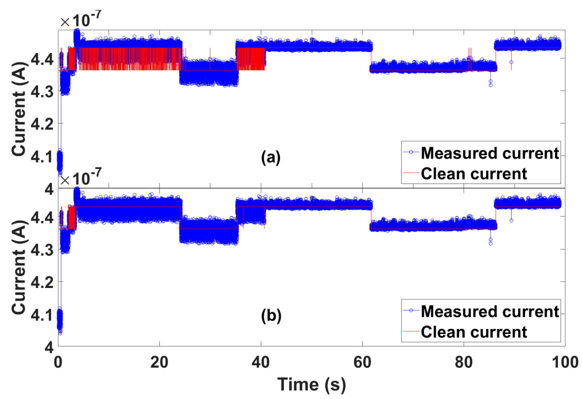

Fig. 4 Comparison between clean current provided by the algorithm without a preprocessing (a) and the same current measurement without the low-pass filter application (b).

7 Conclusion

Machine Learning methods are a technique that is increasingly gaining importance in the scientific world in various ways. This paper has presented a novel technique for applying these types of algorithms to solve the problem of obtaining RTN parameters. We began with the method that had been traditionally used, MLE, observed cases where it does not work, and proposed using the clustering algorithm known as KMeans as a possible solution. Furthermore, in order to achieve a fully automated algorithm for fitting current traces, two different approaches for determining the correct number of clusters in which to group the data have been proposed. We have numerically compared the accuracy of these novel techniques with the one we sought to improve, and in terms of obtaining the correct intensity levels, it is evident that the approach proposed in this paper significantly outperforms MLE.

Clearly, there is still room for improvement in the program. The algorithm is not capable of correctly fitting all the traces that are inputted into it. Some potential avenues for improvement could involve proposing a mixed approach that compares the optimal number of clusters with different methods. In this work, Gap Statistics and Silhouette Score have been presented because they performed the best. Additionally, improving the extraction of capture and emission times through the application of another heuristic method or altering the type of preprocessing could be considered.

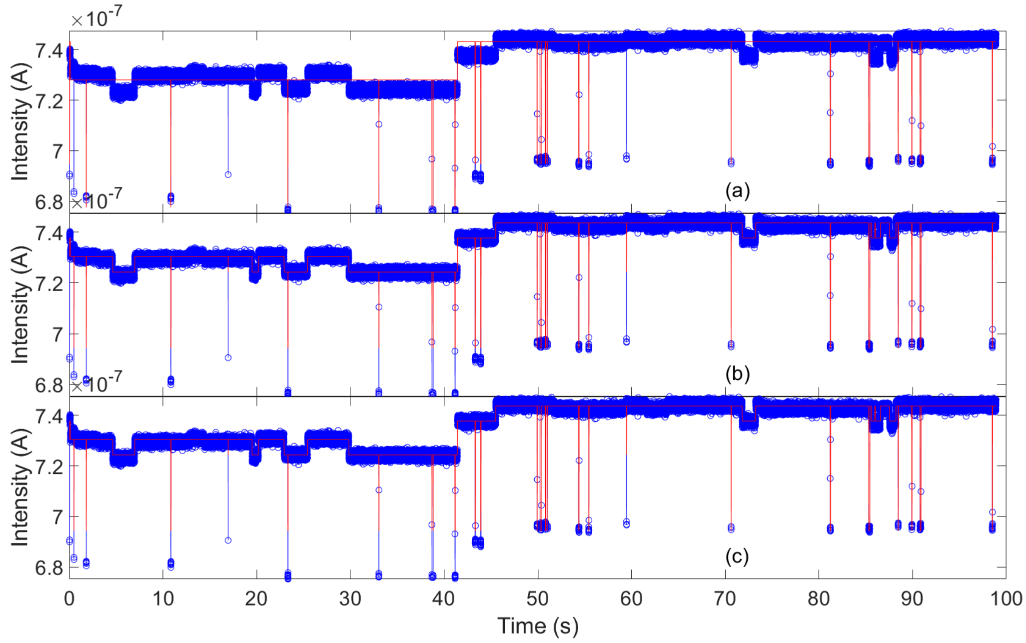

Fig. 5 Results of applying KMeans method combined with Gap Statistics (b) and with Silhouette Score (c) for fitting the trace in Fig.2, which is fitted in (a) using MLE.

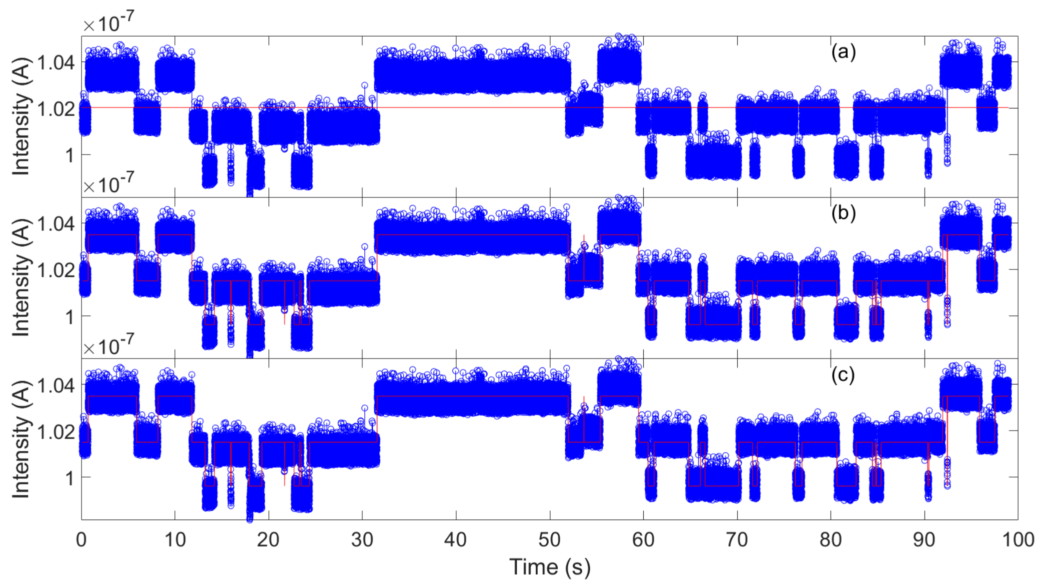

Fig. 6 An example of how a current RTN trace is fitted by (a) Maximum Likelihood Estimation, (b) KMeans with Gap Statistics and (c) KMeans with Silhouette Score. In this example both Gap Statistics and Silhouette Score provides the same number of correct clusters, and therefore KMeans fit it in the same way.

References

[1] Saraza-Canflanca, P., Diaz-Fortuny, J., Castro-Lopez, R., Roca, E., Martin-Martinez, J., Rodriguez, R., Nafria, M., and Fernandez, F., 2020, "A robust and automated methodology for the analysis of Time-Dependent Variability at transistor level," Integration, **72**, pp. 13–20.

[2] MICHL, J., 2022, "Charge Trapping and Variability in CMOS Technologies at Cryogenic Temperatures," Ph.D. thesis, Wien.

[3] Malcolm, A., 2020, "Multi-level Random Telegraph Noise Analysis Using Machine Learning Techniques," Master's thesis, University of Waterloo.

[4] Martin-Martinez, J., Rodriguez, R., and Nafria, M., 2020, "Advanced characterization and analysis of random telegraph noise in CMOS devices," Noise in Nanoscale Semiconductor Devices, pp. 467–493.

[5] Park, H.-S. and Jun, C.-H., 2009, "A simple and fast algorithm for K-medoids clustering," Expert systems with applications, **36**(2), pp. 3336–3341.

[6] Kaur, N. K., Kaur, U., and Singh, D., 2014, "K-Medoid clustering algorithm-a review," Int. J. Comput. Appl. Technol, **1**(1), pp. 42–45.

[7] Li, X., Sun, Y., Wan, J., Chen, B., Cheng, R., and Han, G., 2022, "Machine Learning Method for Accurate Analysis of Complicated Low Temperature Random Telegraph Noise," *2022 International Conference on IC Design and Technology (ICICDT)*, IEEE, pp. 20–23.

[8] Kapila, G. and Reddy, V., 2014, "Impact of sampling rate on RTN time constant extraction and its implications on bias dependence and trap spectroscopy," IEEE Transactions on Device and Materials Reliability, **14**(2), pp. 616–622.

[9] Wirth, G., 2021, "The observation window and the statistical modeling of RTN in time and frequency domain," Solid-State Electronics, **186**, p. 108140.

List of Figures

List of Tables